Curso Python Posgrado: Temario Avanzado

https://gemini.google.com/app/ea2c50ec9aa75674

User prompt: Dame un temario para impartir un curso de lenguaje de programación Python para posgrado

Response: Este programa académico de nivel posgrado está diseñado para profesionales e investigadores con fundamentos previos de programación que buscan dominar Python enfocado en desarrollo de software avanzado, cómputo científico, ciencia de datos y optimización de rendimiento.

Módulo 1: Fundamentos Avanzados y Paradigmas de Programación

- **Modelo de memoria y modelo de objetos de Python:**
  - Manejo de referencias, conteo de referencias y *garbage collection*.
  - Objetos mutables vs. inmutables, *shallow copy* vs. *deep copy*.
  - Modelado de tipos avanzados: `dataclasses`, `NamedTuple` y tipado estático con `typing` (*type hints*, `Protocol`, `Generic`).
- **Programación Orientada a Objetos Avanzada (OOP):**
  - El protocolo de objetos (*Dunder methods* o *magic methods*).
  - Metaprogramación: decoradores con/sin argumentos, inspección en tiempo de ejecución (`inspect`), `__init_subclass__` y metaclases.
  - Inyección de dependencias y patrones de diseño aplicados en Python.
- **Programación Funcional y Generadores:**
  - Funciones de orden superior, iteradores y protocolo de iteración.
  - Expresiones generadoras y funciones generadoras (`yield` / `yield from`).
  - Uso eficiente del módulo `itertools` y `functools` (`lru_cache`, `partial`).

Módulo 2: Arquitectura, Concurrencia y Rendimiento

- **Concurrencia y Paralelismo:**
  - Entendimiento a fondo del *Global Interpreter Lock* (GIL) y su impacto en arquitectura multihilo.
  - Programación multihilo (`threading`, `concurrent.futures`) para tareas *I/O-bound*.
  - Multiprocesamiento (`multiprocessing`) y memoria compartida para tareas *CPU-bound*.

    - Programación asíncrona con `asyncio` (*event loop*, *tasks*, *coroutines* y manejo de recursos asíncronos).

- **Optimización de Código y Perfilado (*Profiling*):**
    - Análisis de complejidad y medición de memoria/tiempo (`cProfile`, `timeit`, `memory_profiler`).
    - Compilación JIT y Cython/Numba para aceleración de algoritmos intensivos.
    - Integración y llamadas C/C++ nativas mediante `ctypes` y `cffi`.

Módulo 3: Ecosistema de Cómputo Científico y Análisis de Datos

- **Vectorización y Cómputo Numérico:**
    - Estructura interna de *arrays* en `NumPy` (*strides*, *data types*, gestión de memoria).
    - Técnicas de *broadcasting*, indexación avanzada y operaciones matriciales sin bucles explícitos.
- **Manipulación de Estructuras de Datos Complejas:**
    - Operaciones de alto rendimiento con `Pandas` y alternativas modernas (`Polars`).
    - Procesamiento e ingeniería de datos a escala: filtrado, agregación, pivoteo y fusiones vectorizadas.
- **Visualización y Comunicación Visual:**
    - Generación de gráficos científicos interactivos y estáticos (`Matplotlib`, `Seaborn`, `Plotly`).

Módulo 4: Ingeniería de Software, Calidad y Despliegue

- **Entornos, Dependencias y Empaquetado:**
    - Aislamiento de entornos virtuales (`venv`, `conda`, `uv`, `poetry`).
    - Creación, distribución y publicación de paquetes y librerías propias (`pyproject.toml`).
- **Pruebas Automatizadas y Calidad de Código:**
    - Estrategias de *testing*: Pruebas unitarias y de integración con `pytest`.
    - *Mocking*, *fixtures* y pruebas de propiedades (`hypothesis`).
    - Análisis estático de código (*linters* y formateadores: `ruff`, `mypy`).
- **Desarrollo de APIs e Interacción con Bases de Datos:**
    - Modelado e interfaces asíncronas para microservicios (`FastAPI`, `Pydantic`).

- Mapeo objeto-relacional (ORM) eficiente y procesamiento masivo con `SQLAlchemy`.

User prompt: En qué libros puedo preparar estos temas

Response: Para estructurar el curso a nivel posgrado con rigor teórico y práctico, la siguiente selección de bibliografía de referencia está categorizada por las áreas clave del temario:

1. Advanced Python, Memory Model & Metaprogramming

- **Fluent Python: Clear, Concise, and Effective Programming** (2ª edición) – *Luciano Ramalho*
  - **Uso en el curso:** Es la referencia definitiva para el **Módulo 1**. Cubre a profundidad el protocolo de objetos (*dunder methods*), *dataclasses*, *type hints* avanzados, generadores, corrutinas y metaprogramación.
- **Python Cookbook** (3ª edición) – *David Beazley & Brian K. Jones*
  - **Uso en el curso:** Excelente como guía de diseño e implementación práctica para patrones de programación orientada a objetos, decoradores complejos y uso de metaclases.

2. Performance, Concurrency & Low-Level Architecture

- **Python High Performance** (3ª edición) – *Gabriele Lanaro & Diego Carver*
  - **Uso en el curso:** Ideal para el **Módulo 2**. Aborda la optimización de código, perfilado de memoria/tiempo (`cProfile`), compilación JIT con Numba y Cython, así como el uso efectivo del GIL.
- **Python Concurrency with asyncio** – *Matthew Fowler*
  - **Uso en el curso:** Enfoque profundo en programación asíncrona, el *event loop*, tareas *I/O-bound* vs *CPU-bound*, y el uso comparativo entre `asyncio`, `multiprocessing` y `threading`.

3. Scientific Computing & Data Analysis

- **Python for Data Analysis** (3ª edición) – *Wes McKinney*
  - **Uso en el curso:** Escrito por el creador de Pandas; vital para el **Módulo 3**. Detalla la estructura interna de `NumPy` (*broadcasting*, *strides*) y la manipulación eficiente de estructuras de datos con `Pandas`.
- **Guide to NumPy** (2ª edición) – *Travis E. Oliphant*
  - **Uso en el curso:** Escrito por el creador de NumPy; útil como lectura complementaria de bajo nivel sobre la gestión de memoria y la interfaz C de arreglos vectorizados.

4. Software Engineering, Testing & Architecture

- **Architecture Patterns with Python** – *Harry Percival & Bob Gregory*
    - **Uso en el curso:** Fundamental para el **Módulo 4**. Enseña cómo aplicar arquitectura limpia, inyección de dependencias, patrones de diseño y modelado de dominio (*DDD*) en proyectos Python complejos.
- **Robust Python: Write Clean, Maintainable Code** – *Patrick Viafore*
    - **Uso en el curso:** Enfocado en la calidad de código, tipado estático avanzado (`mypy`), diseño de interfaces seguras con `Pydantic` y estructuración de pruebas sólidas con `pytest`.
